SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Design and Deployment of a Cloud-Based Virtualized Collaboration Platform for an Engineering College

A PROJECT REPORT

Submitted in the partial fulfilment for the Course of

CSA1507-Cloud Computing And Big Data Analytics

to the award of the degree of

BACHELOR OF ENGINEERING

IN

B. Tech (Artificial Intelligence And Data Science)

Submitted by

Athavan K(192524036)

Under the supervision of

Dr. Anuradha C

September 2026

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

DECLARATION

I Athavan K(192524036) of the B.Tech [AI&DS], Saveetha Institute of Medical and Technical Sciences, Saveetha University, Chennai, hereby declare that the Project Work entitled **Design and Deployment of a Cloud-Based Virtualized Collaboration Platform for an Engineering College** is the result of our own bonafide efforts. To the best of our knowledge, the work presented herein is original, accurate, and has been carried out in accordance with principles of engineering ethics.

Place: CHENNAI

Date:

Athavan K(192524036)

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

BONAFIDE CERTIFICATE

This is to certify that the Project entitled **Design and Deployment of a Cloud-Based Virtualized Collaboration Platform for an Engineering College** has been carried out by Athavan K(192524036) under the supervision of Dr Anuradha C and is submitted in partial fulfilment of the requirements for the current semester of the B. Tech [AI&DS] program at Saveetha Institute of Medical and Technical Sciences, Chennai.

SIGNATURE

Dr Anuradha C

Professor

Department of
Computer Science

SIMATS

Submitted for the Project work Viva-Voce held on _____

INTERNAL EXAMINER **EXTERNAL EXAMINER**

ACKNOWLEDGEMENT

We would like to express our heartfelt gratitude to all those who supported and guided us throughout the successful completion of our Project. We are deeply thankful to our respected Founder and Chancellor, Dr. N.M. Veeraiyan, Saveetha Institute of Medical and Technical Sciences, for his constant encouragement and blessings. We also express our sincere thanks to our Pro-Chancellor, Dr. Deepak Nallaswamy Veeraiyan, and our Vice-Chancellor, Dr. S. Suresh Kumar, for their visionary leadership and moral support during the course of this project.

We are truly grateful to our Director, [Director Name], SIMATS Engineering, for providing us with the necessary resources and a motivating academic environment. Our special thanks to our Principal, Dr Ramesh, for granting us access to the institute's facilities and encouraging us throughout the process.

We are especially indebted to our guide, Dr Anuradha C, for creative suggestions, consistent feedback, and unwavering support during each stage of the project. We also express our gratitude to the Project Coordinators, Review Panel Members, and the entire faculty team for their constructive feedback and valuable inputs that helped improve the quality of our work.

Finally, we thank all faculty members, lab technicians, our parents, and friends for their continuous encouragement and support.

Signature With Student Name

Athavan k(192524036)

ABSTRACT

This report designs and documents a cloud-based virtualized collaboration platform for a small engineering college. The proposed environment consolidates application hosting, file sharing, database services, administration, monitoring, and backup into a logically segmented virtual data centre. A type-2 hypervisor such as VirtualBox is selected for a laboratory-scale demonstration because it is accessible, repeatable, and suitable for showing CPU, memory, storage, and network allocation.

The solution maps institutional requirements to Infrastructure as a Service (IaaS), Platform as a Service (PaaS), and Software as a Service (SaaS). IaaS provides virtual machines, networks, and storage; PaaS provides managed runtimes, databases, and deployment tooling; SaaS provides ready-to-use collaboration capabilities such as document sharing, messaging, and browser-based editing. The design includes a gateway, application VM, storage VM, database VM, monitoring VM, and backup target.

Security is addressed through role-based access control, encrypted transport, network segmentation, patching, backups, logging, and controlled remote administration. The report also defines deployment steps, service tests, resource-utilization observations, cost assumptions, scalability options, limitations, risks, GitHub upload guidance, and learning outcomes aligned to CO1, CO2, CO3 and SDGs 4, 8, and 9.

TABLE OF CONTENTS

The following page plan is used so that the report remains easy to navigate in both printed and digital form. Page numbers refer to the fixed report layout generated with this document.

LIST OF FIGURES

LIST OF TABLES AND ABBREVIATIONS

Abbreviations: API – Application Programming Interface; CPU – Central Processing Unit; DR – Disaster Recovery; IAM – Identity and Access Management; IaaS – Infrastructure as a Service; LAN – Local Area Network; MFA – Multi-Factor Authentication; PaaS – Platform as a Service; RBAC – Role-Based Access Control; RPO – Recovery Point Objective; RTO – Recovery Time Objective; SaaS – Software as a Service; SSH – Secure Shell; TLS – Transport Layer Security; VM – Virtual Machine; VPN – Virtual Private Network.

1. INTRODUCTION AND CONTEXT

Chapter 1 | Requirement analysis

The institution in the problem statement maintains separate physical systems for application hosting, file storage, and user access. This arrangement is understandable when services are acquired one at a time, but it creates fragmented administration. A server can remain underused during normal academic weeks while another service becomes overloaded during examinations, admissions, project reviews, or online events.

The proposed platform treats the data centre as a set of controlled services rather than as a collection of independent machines. Virtualization allows the host to allocate isolated CPU, memory, virtual disks, and network interfaces to multiple workloads. Cloud service models add a second perspective: the institution can operate infrastructure itself, consume a managed platform, or use a ready-made application depending on how much control and administration it needs.

This report therefore combines analysis and application. It first identifies requirements and maps them to IaaS, PaaS, and SaaS. It then defines an implementable virtualized architecture, gives deployment steps, specifies service communication and access controls, and evaluates the result against cost, utilization, scalability, accessibility, security, collaboration, and operational limitations.

- Primary goal: consolidate academic digital services into a manageable virtualized environment.
- CO1 emphasis: classify and justify cloud service models for institutional requirements.
- CO2 emphasis: design and deploy multiple VMs with resources and network connectivity.
- CO3 emphasis: demonstrate secure access and collaboration within cloud services.

1.1 PROBLEM DEFINITION, AIM AND OBJECTIVES

Problem definition: the college needs online learning, file sharing, application hosting, and collaborative academic activities, but its separate physical systems lead to high cost, inefficient resource utilization, and difficulty handling variable demand. The solution must demonstrate cloud concepts in a small, observable environment rather than merely describe them.

Aim: to design and document a cloud-based virtualized collaboration platform that provides a secure foundation for academic applications and shared resources while remaining suitable for a laboratory demonstration.

The design objectives are deliberately measurable. A successful implementation should boot the planned VMs, assign resources according to workload, establish communication on the intended networks, expose only approved services, support role-based activities, create backup copies, and produce evidence that can be inspected by an evaluator.

- Identify users, workloads, functional requirements, quality requirements, constraints, and assumptions.

- Compare IaaS, PaaS, and SaaS using control, responsibility, cost, scalability, accessibility, and security.
- Create a virtualized data-centre architecture with application, storage, database, monitoring, and backup roles.
- Define secure cloud access through HTTPS, SSH administration, least privilege, and optional VPN.
- Evaluate the design against the assignment rubric and document reproducible evidence.

1.2 INSTITUTIONAL REQUIREMENTS

Requirements are grouped into functional, operational, and quality categories. Functional requirements describe what users must be able to do; operational requirements describe what administrators must operate; quality requirements describe how well and how safely the platform should perform.

The design assumes a small college with a limited IT team, a mixed population of students and faculty, a campus LAN, an internet connection, and a laboratory host capable of running several lightweight Linux VMs. The reference sizing is intentionally conservative. It can be enlarged after observing actual demand, but it should not begin with an unnecessarily complex enterprise topology.

ID	REQUIREMENT	PRIORITY	ACCEPTANCE INDICATOR
R1	Browser access to learning application	High	Student can sign in and view a course page
R2	Shared academic files with version awareness	High	Authorized user uploads and retrieves a file
R3	Collaborative editing and communication	High	Faculty and students share comments or edits
R4	Separate administrative access	High	Only admin role can change platform settings
R5	Remote management	Medium	Admin connects through SSH or VPN path
R6	Backup and recovery	High	Backup job creates a restorable copy
R7	Observability	Medium	CPU, memory, disk and service health are logged

1.3 STAKEHOLDERS, WORKLOADS AND ASSUMPTIONS

The same platform is experienced differently by each stakeholder. Students value availability, simple browser access, and safe submission of work. Faculty value course management, controlled sharing, and collaboration. Administrators value repeatable deployment, logs, backups, and a clear boundary between public and private services. Institutional leadership values predictable cost, capacity growth, and reduced infrastructure waste.

The workload pattern is bursty: most activity is moderate, but assignment deadlines and examinations create short periods of high concurrency. This is a strong reason to use virtualized allocation and cloud-style elasticity rather than permanently dedicating one physical server to every service.

STAKEHOLDER	MAIN WORKLOAD	RISK IF UNSUPPORTED	DESIGN RESPONSE
Students	View content, upload work, collaborate	Missed deadlines or inaccessible resources	HTTPS application and storage
Faculty	Create, review, grade, share	Slow feedback and duplicated files	Role-aware application and versions
Administrators	Provision, patch, monitor, restore	Long outages and unclear ownership	Separate management path and runbook
Management	Budget and capacity oversight	Unplanned spend and idle hardware	Sizing model and utilization metrics
Auditor / evaluator	Verify outcomes and controls	Low confidence in implementation	Test plan and evidence checklist

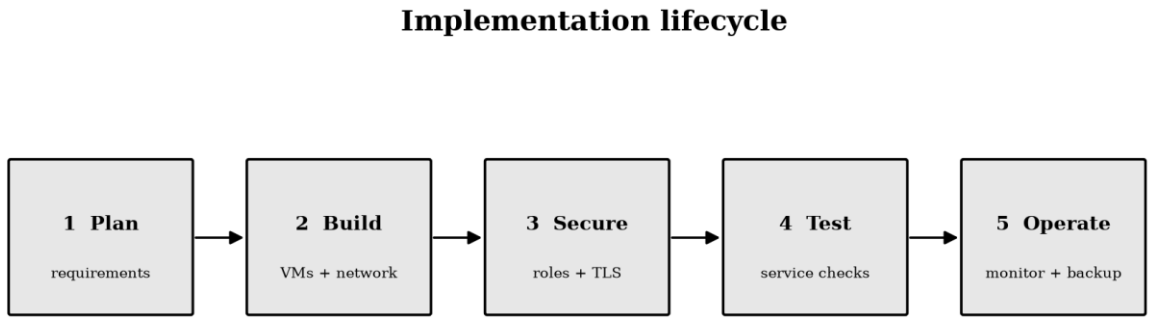

Figure 6. Implementation lifecycle used to move from requirements to operation.

2. CLOUD SERVICE MODELS

A cloud service model describes which layer is consumed and which party is responsible for operating the layers below it. In IaaS, the consumer receives virtualized compute, networking, and storage and remains responsible for the guest operating system and applications. In PaaS, the provider also operates the runtime and much of the platform stack. In SaaS, the consumer primarily configures and uses a complete application.

The models are not competing labels for the same purchase. They represent different responsibility boundaries. A college may use IaaS for a custom application, PaaS for a database or deployment pipeline, and SaaS for messaging or collaborative documents at the same time. A hybrid design often produces the best balance between control and administrative effort.

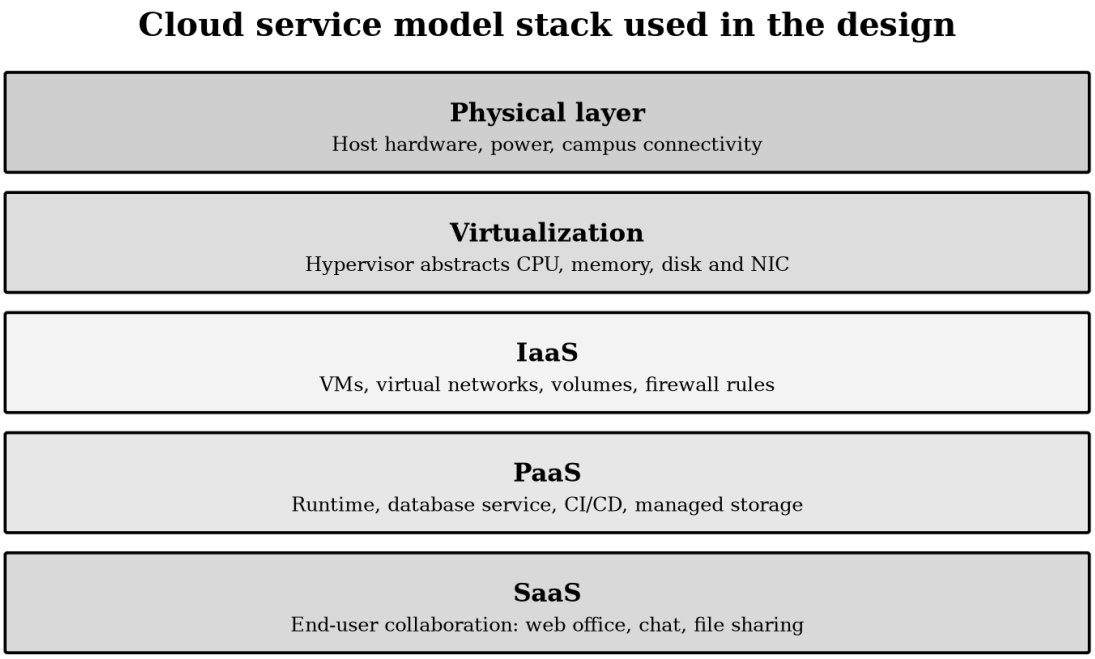

Figure 2. Cloud service model stack applied to the proposed platform.

2.1 INFRASTRUCTURE AS A SERVICE (IAAS)

IaaS supplies configurable infrastructure resources: virtual machines, virtual networks, virtual disks, firewalls, load balancers, and snapshots. In the reference implementation the local hypervisor represents an IaaS-like layer. The college controls VM images, operating systems, service packages, internal addressing, and resource allocation.

IaaS is suitable when the institution has a custom academic application, needs operating-system access, or wants to learn virtualization directly. It is also useful for development and testing because VMs can be cloned, paused, snapshotted, and reconfigured without purchasing new physical servers.

The trade-off is responsibility. The college must patch guests, configure firewall rules, monitor storage, protect credentials, test backups, and recover failed services. IaaS therefore provides high control but also a large operational surface. The report limits

the initial footprint to small Linux VMs and documents the tasks that an administrator must own.

IAAS BENEFIT	HOW IT HELPS THE COLLEGE	LIMITATION / CONTROL
Control	Custom OS, packages, ports and policies	Requires disciplined administration
Isolation	Separate workloads into VMs	A compromised host affects many guests
Flexibility	Resize or clone a VM for a new course	Sizing must be measured, not guessed
Recoverability	Snapshots and image-based rebuilds	Snapshots are not a complete backup

2.2 PLATFORM AS A SERVICE (PAAS)

PaaS supplies a managed application platform such as a runtime, database, object store, message queue, or continuous deployment service. The consumer focuses on code, data, and configuration while the provider handles many operating-system and platform tasks. Examples relevant to this scenario include a managed PostgreSQL service, container application platform, or hosted CI/CD pipeline.

PaaS can reduce patching work and improve repeatability. A development team can deploy the same application through a build pipeline, use managed backups, and scale a service without manually rebuilding operating systems. This is especially attractive when the college wants to host a custom portal but has limited infrastructure staff.

The disadvantages are reduced low-level control, provider-specific configuration, possible usage-based charges, and migration effort if APIs or deployment formats become tightly coupled to one provider. The report recommends PaaS for new application runtimes and managed databases when the institution accepts the provider's operating boundary.

- Good fit: application runtime, managed database, object storage, CI/CD, background jobs.
- Responsibility retained: application code, data classification, identities, permissions, and budget controls.
- Evaluation question: can the service be exported or rebuilt if the provider becomes unavailable?

2.3 SOFTWARE AS A SERVICE (SAAS)

SaaS is a complete application delivered through a browser or client. The provider operates the infrastructure, platform, application updates, and much of the availability mechanism. Users configure accounts, groups, sharing permissions, retention, and collaboration settings.

For a college, SaaS is a strong fit for email, calendar, chat, video meetings, office documents, and routine file collaboration. It delivers value quickly and avoids the need to operate every component of a communication stack. The institution should still

evaluate data location, retention, identity integration, export options, accessibility, and contractual responsibility.

SaaS does not eliminate security work. A user can still overshare a file, reuse a password, or grant a public link. Administrative controls, role design, user training, MFA, audit logs, and offboarding remain important. The proposed platform uses SaaS where the requirement is a standard collaboration capability rather than a custom learning service.

SAAS USE	VALUE	CONTROL TO CONFIGURE
Email and calendar	Reliable communication and scheduling	Groups, retention, MFA
Document collaboration	Concurrent editing and comments	Share scope, version history
Chat / meetings	Fast academic communication	Guest access and moderation
Hosted storage	Access from multiple devices	Quota, link expiry, recovery

2.4 COMPARATIVE ANALYSIS OF SERVICE MODELS

The comparison below evaluates the models against the criteria requested in the assignment. The ratings are qualitative and are meant to support a design decision rather than claim a universal ranking. A managed SaaS product can be more scalable than a self-managed IaaS deployment, but it may provide less customization. Conversely, IaaS can be cheaper at small steady utilization but more expensive in staff time.

CRITERION	IAAS	PAAS	SAAS
Consumer control	High: VM and OS control	Medium: code/configuration control	Low: application configuration
Administrative effort	High	Medium	Low to medium
Customization	High	High within runtime limits	Low to medium
Elasticity	Depends on design	Usually strong	Provider-managed
Cost pattern	Capacity + operations	Usage + platform	Subscription / user
Security responsibility	Shared, more consumer work	Shared, platform helps	Provider + customer configuration
Best use here	Custom academic services	Runtime and database	Communication and editing

NOTE: *A service model decision should be revisited when enrollment, compliance needs, data volume, or staffing changes.*

2.5 REQUIREMENT-TO-MODEL MAPPING

Mapping requirements to service models avoids using the same delivery pattern for every problem. The proposed solution deliberately mixes models. The virtualized laboratory demonstrates IaaS concepts, a containerized or managed runtime represents PaaS, and a browser-based collaboration suite represents SaaS. This combination reflects how real institutions assemble a portfolio of services.

INSTITUTIONAL NEED	PRIMARY MODEL	RATIONALE	EXAMPLE
Custom LMS or API	IaaS / PaaS	Needs custom logic; platform can reduce operations	Application VM or managed runtime
Database for course metadata	PaaS	Managed patching and backups are valuable	Managed PostgreSQL
Shared files	SaaS or IaaS	SaaS is simple; self-hosting gives control	Hosted drive or Nextcloud
Email, calendar, chat	SaaS	Commodity service; availability matters	Education collaboration suite
VM administration	IaaS	Demonstrates allocation and virtualization	VirtualBox / KVM
Monitoring and backup	IaaS / PaaS	Operational control and recovery	Monitoring VM + backup target

2.6 DEPLOYMENT MODEL AND DESIGN PRINCIPLES

The demonstration is a private, laboratory-scale cloud environment because the VMs are operated on an institutional or personal host under the student's control. In production, the same service roles could be placed in a public cloud, private cloud, or hybrid arrangement. A hybrid option would keep identity and sensitive academic records under institutional control while consuming SaaS for collaboration.

Six principles guide the design: least privilege, segmentation, repeatability, observability, recoverability, and progressive scaling. These principles are more durable than any single product choice. They also make the report transferable to other hypervisors or cloud providers.

- Least privilege: expose only required ports and give each role only the permissions it needs.
- Segmentation: keep public entry, service traffic, management traffic, and backups logically distinct.
- Repeatability: use documented VM names, addresses, package lists, and test commands.
- Observability: measure CPU, memory, disk, service health, login events, and backup status.
- Recoverability: define restore order, RPO, RTO, and evidence that a backup can be read.

3. VIRTUALIZED DATA-CENTRE ARCHITECTURE

The architecture separates user-facing access from internal service communication. Users reach the gateway over HTTPS. The gateway forwards approved requests to the application layer, while the storage and database services remain on private networks. Administration uses a restricted management path, and backups are sent to a separate target rather than being treated as ordinary application storage.

A single physical host is sufficient for a demonstration, but it is a single point of failure. Production would use multiple hosts, redundant storage, monitored power, and a tested disaster-recovery plan. The report explicitly distinguishes what the laboratory proves from what a production deployment would require.

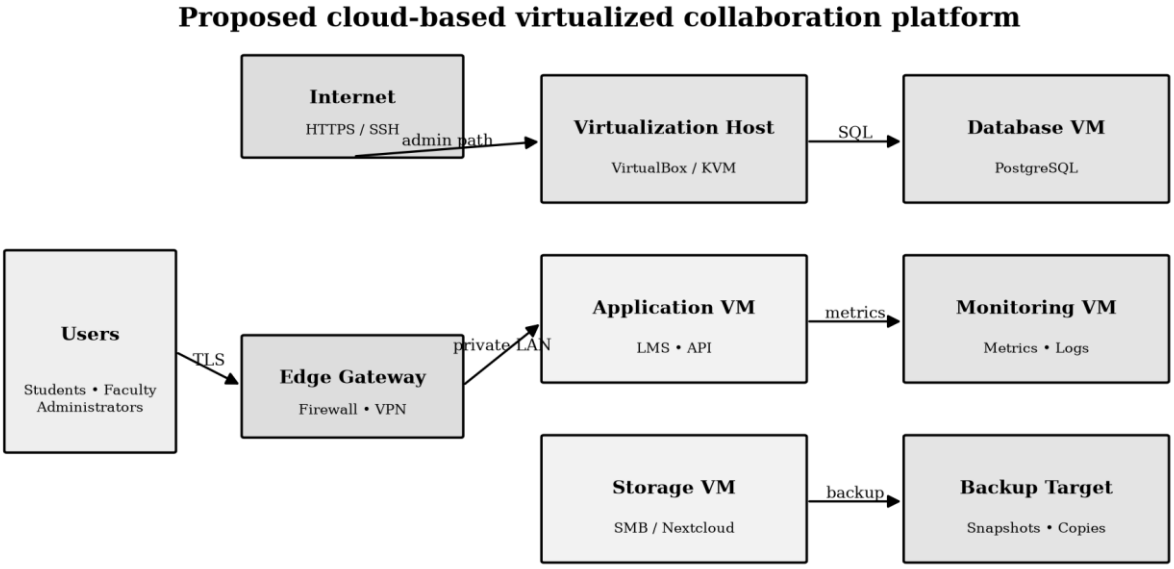

Figure 1. Reference architecture for the cloud-based virtualized collaboration platform.

3.1 VIRTUAL MACHINE INVENTORY AND RESOURCE PLAN

The VM inventory is sized for a small demonstration and can be adjusted to the host's capacity. The storage values are virtual disk allocations, not a guarantee of physical free space. Dynamic disks should be monitored because over-allocation can create a host-level failure even when a guest still reports free space.

VM	ROLE	VCPU	RAM	DISK	NETWORK
GW-01	Reverse proxy, firewall, VPN entry	1	1 GB	12 GB	DMZ + mgmt
APP-01	Learning application and API	2	2 GB	25 GB	Service + mgmt
DB-01	PostgreSQL metadata database	2	2 GB	30 GB	Service + backup
FILE-01	File sharing and versioned documents	2	2 GB	60 GB	Service + backup
MON-	Metrics, logs and	1	1 GB	20 GB	Mgmt

01	health checks				
BKP-01	Backup target / restore staging	1	1 GB	80 GB	Backup + mgmt

NOTE: *Minimum demonstration host: approximately 8 vCPU threads, 10–12 GB RAM available to guests, and 250 GB free storage. Reduce RAM or run fewer VMs if the host is smaller.*

3.2 VIRTUALIZATION CONCEPTS

Virtualization inserts a hypervisor between physical hardware and guest operating systems. The hypervisor schedules vCPUs on physical cores, maps guest memory to host memory, stores guest disks as files or logical volumes, and connects virtual NICs to software switches. Each VM behaves as an independent computer from the guest's perspective.

The key benefit for this assignment is consolidation: multiple services can share one host while remaining separately manageable. Isolation is not absolute; a host outage can stop every guest, and incorrect virtual networking can prevent service communication. Therefore, virtualization must be accompanied by host patching, resource monitoring, and backup of VM definitions and data.

Snapshots are useful before a risky configuration change, but they should not replace backups. A snapshot depends on the base disk and can grow over time. A backup is a separate copy that can be restored after disk corruption, accidental deletion, or host failure.

- Compute abstraction: vCPU scheduling and CPU caps.
- Memory abstraction: assigned RAM, ballooning or swapping, and host headroom.
- Storage abstraction: virtual disks, dynamic allocation, I/O performance, and backup.
- Network abstraction: NAT for outbound access, bridge for LAN visibility, and host-only or internal networks for isolation.

3.3 NETWORK TOPOLOGY AND ADDRESSING

The network plan uses logical separation instead of placing every interface on one flat segment. The DMZ contains the reverse proxy or gateway entry point. The service LAN carries application-to-database and application-to-file traffic. The management LAN carries SSH, metrics, and administrative operations. The backup path is restricted to backup-capable hosts.

SEGMENT	CIDR	ALLOWED TRAFFIC	CONTROL
DMZ	172.16.10.0/24	HTTPS to gateway; gateway to app	Firewall / reverse proxy
Service LAN	172.16.20.0/24	App to DB and file service	Private interface rules
Management LAN	172.16.30.0/24	SSH, metrics, admin tools	Admin source allowlist
Backup path	172.16.40.0/24	Backup push / pull only	Key-based access

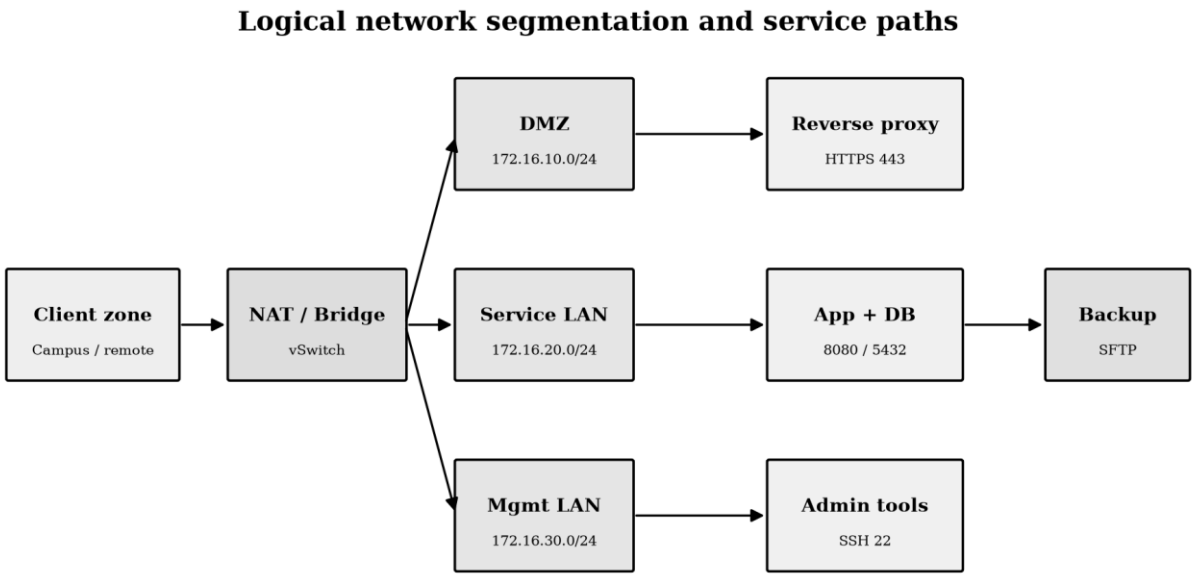

Figure 3. Logical network segmentation and service paths.

4. DEPLOYMENT METHOD

The deployment follows a controlled sequence: prepare the host, create a base Linux image, clone role-specific VMs, assign resources, configure networks, install services, apply security controls, test communication, and capture evidence. Each stage has a rollback point and a validation command.

For a student demonstration, VirtualBox is an accessible option. KVM/libvirt is a strong Linux-native alternative. The report uses product-neutral concepts first and gives representative Linux commands second. The exact GUI labels may vary by version, so the evaluator should focus on the resource values, names, interfaces, addresses, and results.

- Prepare a host-only or internal network and record the chosen CIDR ranges.
- Create the base Ubuntu Server VM with updates, an administrative user, and SSH.
- Clone the base VM into GW-01, APP-01, DB-01, FILE-01, MON-01, and BKP-01.
- Assign each clone a fixed hostname, static service address, and role-specific disk.
- Install only the packages required for the role, then apply firewall rules.
- Run functional tests before taking final screenshots.

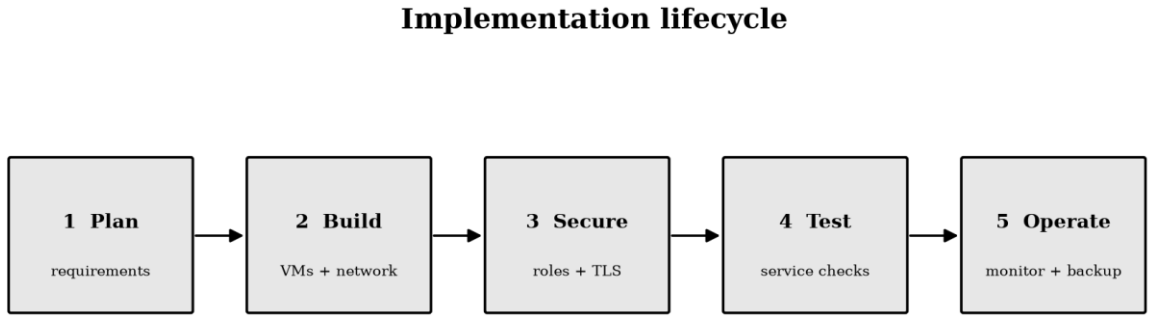

Figure 6. The implementation lifecycle should be followed in order.

4.1 BASE VM AND HOST PREPARATION

Host preparation begins with a resource check. Hardware virtualization must be enabled in firmware, the hypervisor must be installed, and enough RAM and disk must remain for the host operating system. The base VM should use a supported 64-bit Linux distribution, a stable update channel, and a separate administrative account rather than routine root login.

A consistent naming convention simplifies logs and screenshots. The pattern ROLE-NN is used throughout this report. The base image is updated before cloning so that every guest begins with the same security baseline; role-specific packages are installed only after cloning.

```
# Verify host and guest identity
hostnamectl
ip addr
free -h
df -h

# Update the base guest
sudo apt update
sudo apt full-upgrade -y
sudo apt install -y openssh-server curl git ufw qemu-guest-agent

# Create a non-root administrator
sudo adduser cloudadmin
sudo usermod -aG sudo cloudadmin
```

NOTE: *The commands are representative. Capture terminal screenshots from the actual VM after successful execution; do not submit a code block as proof of a live result.*

4.2 VM CREATION AND RESOURCE CONFIGURATION

Each VM is created from the base image, given a role-specific hostname, and connected to the correct virtual networks. CPU and memory are assigned from the inventory table rather than arbitrarily. The host should retain sufficient headroom for its own operation; allocating every host core or every gigabyte to guests makes the system fragile.

The resource plan separates compute-heavy application and database services from file and monitoring roles. In a small host, the monitoring VM can be combined with the gateway, but keeping it separate produces clearer evidence of service isolation. A production environment would consider high availability, live migration, storage IOPS, and autoscaling.

CHECK	EXPECTED CONFIGURATION	EVIDENCE
VM name	GW-01, APP-01, DB-01, FILE-01, MON-01, BKP-01	Hypervisor inventory screenshot
CPU / RAM	Match Table 4; host keeps headroom	Resource settings screenshot
Disk	OS disk plus role data disk where required	Storage settings screenshot
NICs	Correct segment and adapter mode	Network settings screenshot
Boot	VM starts cleanly and receives expected address	Console + ip addr screenshot
Guest tools	Time sync and shutdown integration enabled	Guest tools status

4.3 OPERATING SYSTEM AND SERVICE BASELINE

After cloning, every VM receives a unique hostname and address. The administrator disables unnecessary services, enables automatic security updates where appropriate, and configures the host firewall with a default-deny posture. Service accounts are used for applications so that a web process does not operate with unrestricted administrator privileges.

The baseline is intentionally small. A smaller package footprint reduces attack surface and makes troubleshooting easier. The baseline also provides a common evidence point: the evaluator can compare the hostname, IP configuration, listening ports, service status, and update state across VMs.

```
# On each cloned VM
sudo hostnamectl set-hostname APP-01
sudo systemctl enable --now ssh
sudo ufw default deny incoming
sudo ufw default allow outgoing
sudo ufw allow from 172.16.30.0/24 to any port 22 proto tcp
sudo ufw enable
sudo ss -tulpn
systemctl --failed

# Verify the identity and address
hostnamectl --static
ip -br addr
```

NOTE: *Replace APP-01 and the allowed source range for each role. Never open database or management ports to the public internet.*

4.4 APPLICATION, DATABASE AND FILE SERVICES

The application service provides course pages, authentication integration, assignment metadata, and collaboration links. The database stores structured metadata such as users, courses, submissions, and permissions; it should not be exposed directly to end users. The file service stores documents and version history, with quotas and group permissions applied at the application layer.

A clean separation lets the database and file service be backed up independently and allows the application to be rebuilt from code and configuration. In a PaaS deployment, the application runtime and database may be managed by the provider; in this IaaS demonstration, the responsibilities remain visible inside the VMs.

SERVICE	LISTENING ROLE	DATA	OWNER
Reverse proxy	Public HTTPS entry	Certificates and routing	Administrator
Application API	Course and workflow logic	Configuration + metadata	Application team
PostgreSQL	Private database	Structured records	Database administrator
File platform	Private or proxied file access	Documents and versions	Faculty / admin

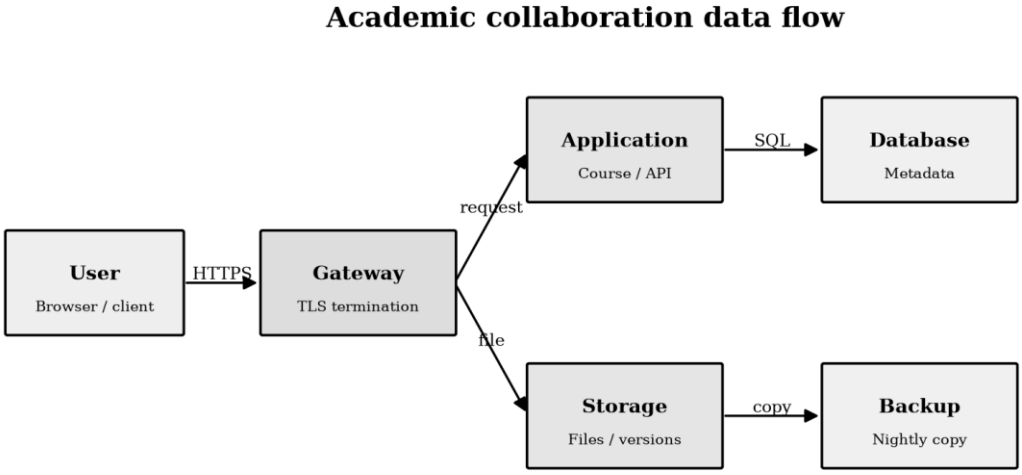

Figure 7. User requests are routed through the gateway and then to application, storage and database services.

4.5 COLLABORATION SERVICE CONFIGURATION

Collaboration is treated as a set of controlled actions rather than as an unrestricted shared folder. A faculty member creates a course workspace, adds students through a group, shares a document with the group, and uses comments or version history for review. Students can submit work without changing the original teaching material. Administrators retain recovery and audit capability.

A SaaS office suite can provide browser editing, chat, calendar, and notifications with minimal infrastructure effort. A self-hosted file platform can provide stronger local control and a useful IaaS demonstration. The recommended design is to select one primary file collaboration service and integrate links from the learning application instead of duplicating files across systems.

COLLABORATION ACTION	STUDENT	FACULTY	ADMIN
View course material	Allowed	Allowed	Allowed
Upload assignment	Own submission	Course workspace	Recovery only
Edit shared document	Group permission	Owner / editor	Emergency support
Comment / review	Allowed where enabled	Allowed	Audit view
Delete / restore	Own draft only	Course content policy	Controlled restore
Invite external user	Not allowed	Policy approval	Policy owner

4.6 CONFIGURATION MANAGEMENT AND REPEATABILITY

Repeatability is the difference between a one-time demonstration and an infrastructure design. Configuration should be recorded in a README, an inventory file, and role-specific scripts. Secrets must not be committed to GitHub; use placeholders, environment variables, or a secret manager. VM images should be versioned by date or release label, and changes should be made through a documented procedure.

A practical repository separates documentation from automation. The documentation explains architecture and decisions. Shell scripts or Ansible playbooks express installation and configuration. A test script checks DNS or host resolution, TCP reachability, HTTP status, database readiness, storage write access, and backup file creation.

REPOSITORY PATH	PURPOSE	SECRET-SAFE CONTENT
docs/	Architecture, diagrams, runbook	Addresses and examples only
infra/	VM notes, network plan, firewall rules	No private keys
scripts/	Install and test scripts	Read secrets from environment
evidence/	Screenshots and result index	Redact usernames and tokens
README.md	How to reproduce and assess	Public-safe setup steps

4.7 EVIDENCE CAPTURE DURING DEPLOYMENT

Evidence should be captured at the moment a stage succeeds. A screenshot is strongest when it shows the VM name, the relevant command or setting, and an unambiguous result. Screenshots should be numbered and referenced in the report. Avoid capturing passwords, private keys, session tokens, personal student information, or public URLs that should remain private.

The evidence plan below is designed to satisfy the assignment's screenshots/evidence deliverable without turning the report into an unstructured gallery. Each screenshot answers one verification question and is paired with a short explanation of what the evaluator should observe.

EVIDENCE ID	CAPTURE	VERIFIES
E-01	Hypervisor VM inventory	All planned VMs exist
E-02	VM resource settings	CPU, memory and disk allocation
E-03	Network adapters / ip addr	Correct segments and addresses
E-04	systemctl + ss output	Services are running and ports are intentional
E-05	Browser login and course page	Application access works
E-06	Shared file upload and version	Collaboration workflow works
E-07	Ping / curl / nc tests	VM-to-VM communication works
E-08	Backup directory and restore test	Recovery path is usable

NOTE: *This report provides the evidence structure. Replace each evidence item with an authentic screenshot from the completed deployment.*

5. CLOUD ACCESS AND REMOTE RESOURCE MANAGEMENT

Cloud access is the path through which a user or administrator reaches a resource without being physically at the host. User access should terminate at HTTPS or an approved SaaS identity provider. Administrative access should use SSH keys, a restricted management subnet, and preferably a VPN or bastion host. Direct public exposure of SSH, database ports, or hypervisor management interfaces is avoided.

Remote resource management includes starting and stopping VMs, reading service status, viewing logs, checking disk capacity, applying updates, and restoring backups. These actions are privileged and must be logged. A short runbook should state who can perform them, what to check before a change, and how to roll back.

- Users: HTTPS browser access, session timeout, MFA where available.
- Administrators: SSH key authentication, source allowlist, sudo audit, no shared accounts.
- Operators: dashboards, health checks, alert acknowledgement, backup verification.
- Emergency path: documented console access when network management is unavailable.

5.1 IDENTITY, ROLES AND PERMISSIONS

Role-based access control maps a person's role to a set of permitted actions. It avoids creating one-off permissions for every user and makes offboarding more reliable. A student should not be able to read another student's private submission; a faculty member should manage only the courses assigned to them; an administrator can operate the platform but should still use a named account.

ROLE	APPLICATION	FILES	INFRASTRUCTURE
Student	View enrolled courses; submit work	Read shared; write own submission	No access
Faculty	Create and grade course activities	Manage course workspace	No direct DB access
Administrator	Manage configuration and users	Restore with approval	VM and service operations
Auditor	Read reports and audit logs	Read evidence only	Read-only monitoring

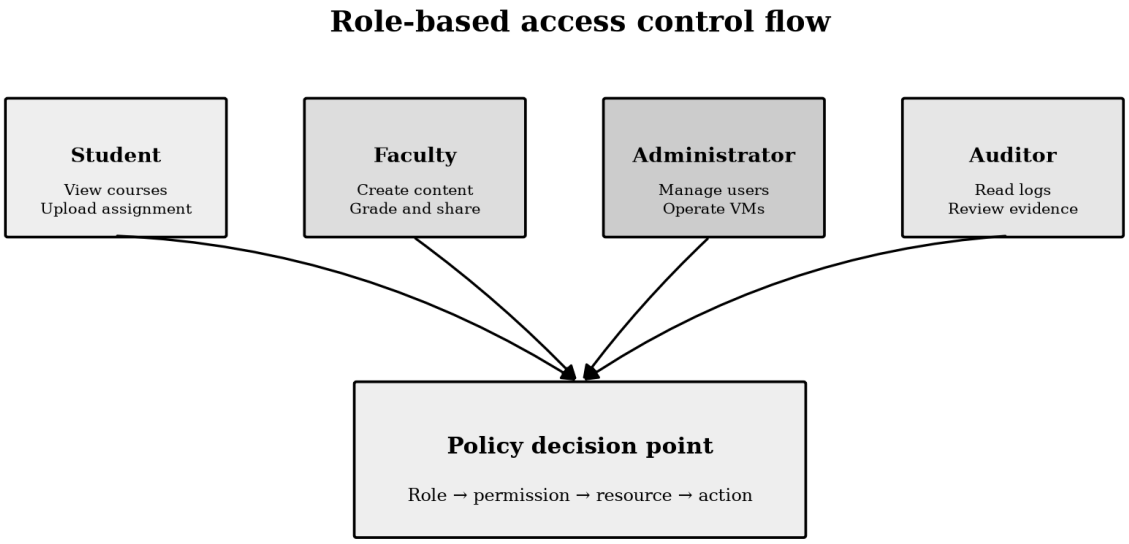

Figure 5. Role-to-permission decision flow.

5.2 SECURITY CONTROLS AND THREAT MODEL

The threat model considers accidental as well as malicious events. A leaked password, an over-broad sharing link, a vulnerable package, a full disk, or a failed backup can harm availability and confidentiality even without an advanced attacker. Controls are therefore layered and prioritized according to the institution's capacity.

THREAT	IMPACT	PRIMARY CONTROL	SECONDARY CONTROL
Credential theft	Account misuse	MFA and strong unique credentials	Login alerts and session revocation
Public file link	Data disclosure	Default-private sharing	Expiry, audit and user training
Unpatched guest	Service compromise	Patch schedule and minimal packages	Network segmentation
Host disk failure	Data unavailability	Separate backup target	Restore drill and RPO
Ransomware / deletion	Data loss	Immutable or offline copy	Retention and recovery test

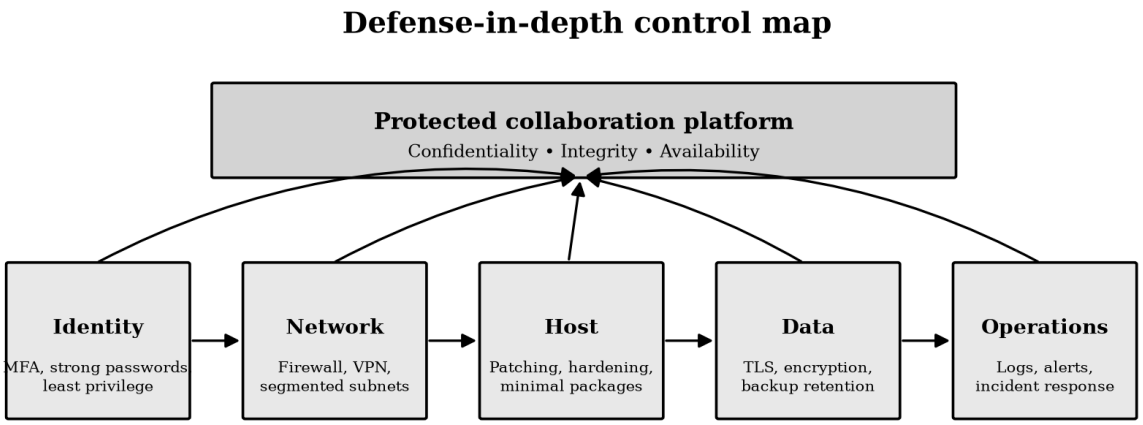

Figure 4. Defense-in-depth controls.

5.3 BACKUP, RECOVERY AND OPERATIONS

The backup design separates configuration, application data, database dumps, and user files. A nightly database dump is useful for structured recovery; a file-level or snapshot-based copy protects documents; VM definitions and firewall rules make the environment reproducible. At least one copy should be isolated from the main host so that a host compromise does not destroy every copy.

Recovery objectives make the design testable. A demonstration target of RPO 24 hours and RTO 4 hours is reasonable for a small college platform, but the institution should set its own values. The restore order is gateway, database, file service, application, monitoring, and then user validation.

ITEM	TARGET	VERIFICATION
Database	Daily dump; retain 7–14 days	Restore into staging and query a known record
Academic files	Daily incremental + weekly full	Open a restored sample and check versions
VM configuration	After every approved change	Recreate or import on a clean host
Secrets / certificates	Protected separately	Validate expiry and controlled recovery
Monitoring data	Short retention acceptable	Confirm alerts after restore

```
# Representative database backup
pg_dump -Fc -h DB-01 -U appuser learning >
/backup/learning_YYYYMMDD.dump

# Check that the file exists and is readable
ls -lh /backup/
sha256sum /backup/learning_YYYYMMDD.dump

# Restore into a staging database (example)
createdb -h DB-01 -U appuser learning_restore
pg_restore -h DB-01 -U appuser -d learning_restore
/backup/learning_YYYYMMDD.dump
```

6. TESTING STRATEGY

Testing is organized by layer: infrastructure, network, service, security, collaboration, recovery, and resource behavior. A test has a clear precondition, action, expected result, actual result, and evidence identifier. This structure prevents a report from claiming success merely because a VM booted.

The reference test plan below should be executed after configuration. Mark each test PASS or FAIL, record the date, and link the screenshot or terminal output. Failed tests should remain visible with their corrective action because troubleshooting is part of an engineering evaluation.

ID	TEST	EXPECTED RESULT	EVIDENCE
T01	Boot all VMs	All guests start without service failure	E-01
T02	Reach gateway over HTTPS	Valid page or login response	E-05
T03	App to database port	Connection succeeds only on service LAN	E-07
T04	Student permission check	Cannot access another user’s private file	E-06
T05	Faculty collaboration	Can share and review course document	E-06
T06	Admin SSH restriction	Only approved source can connect	E-03
T07	Backup creation	Timestamped readable backup exists	E-08
T08	Restore sample	Known file or record is recovered	E-08
T09	Disk alert	Low-space condition is visible	E-04

6.1 INFRASTRUCTURE AND NETWORK TEST RESULTS

Infrastructure tests validate the foundation before users are introduced. The evaluator should confirm that every VM has the expected hostname, address, CPU, memory, and disk. Network tests then validate that allowed paths work and prohibited paths fail. A secure result includes both positive and negative tests.

TEST COMMAND / ACTION	PASS CONDITION	INTERPRETATION
hostnamectl / ip -br addr	Name and address match inventory	Identity is deterministic
free -h / df -h	Resources are available with headroom	No immediate pressure
ping or curl on service path	Expected host responds	Routing and service are reachable
nc -zv DB-01 5432	Only APP-01 or admin path succeeds	Database is not public
curl from unauthorized segment	Connection rejected or filtered	Segmentation is effective

systemctl --failed	No unexpected failures	Guest baseline is healthy

```
# Illustrative connectivity checks
ping -c 3 172.16.20.11
curl -I https://platform.example.invalid
nc -zv 172.16.20.12 5432
sudo ufw status verbose
systemctl --failed
```

NOTE: *Use the actual internal addresses and domain name in your environment. The .invalid domain above is intentionally non-routable documentation text.*

6.2 SERVICE AND COLLABORATION TEST RESULTS

Service tests follow the academic workflow from sign-in to recovery. A successful login alone is insufficient: the application must read course metadata, create or retrieve a file, apply the correct permission, and record a useful audit event. Collaboration tests should include at least one student and one faculty account or clearly documented test identities.

SCENARIO	STEPS	EXPECTED RESULT
Course access	Sign in as student; open enrolled course	Course content loads over HTTPS
Assignment submission	Upload a small file; view own submission	File is stored and visible to owner
Privacy	Attempt a second student's private URL	Access is denied
Faculty review	Sign in as faculty; comment or grade	Authorized review action succeeds
Versioning	Edit shared document twice	Previous version can be identified
Notification	Create a collaboration event	Configured recipient receives event
Restore	Delete a test file and restore it	File is recovered with correct ownership

6.3 RESOURCE UTILIZATION AND PERFORMANCE

Resource utilization should be measured rather than described as efficient by assumption. Record idle and peak CPU, memory consumption, disk growth, and response time during a small concurrent test. The purpose is not to claim enterprise benchmark performance; it is to show that resources are visible and that the architecture can be resized using evidence.

A useful demonstration has a baseline period, a workload period, and a recovery period. For example, record five minutes at idle, upload several test files and open concurrent sessions, then observe the return to baseline. If memory pressure, swap, disk

latency, or queue growth appears, identify the bottleneck and propose a targeted change.

METRIC	BASELINE	TEST WINDOW	DECISION RULE
vCPU utilization	Record per VM	During concurrent access	Resize if sustained > 75%
Memory used	Record working set	During uploads / queries	Add RAM or reduce services if swapping
Disk used	Record percentage	After file and backup tests	Alert before 80%; act before 90%
HTTP response	Record median / max	During representative pages	Investigate high tail latency
Backup duration	Record elapsed time	Nightly or manual run	Must finish before next window

6.4 SCALABILITY ANALYSIS

Scalability means increasing capacity without redesigning every service. Vertical scaling adds CPU, memory, or disk to a VM and is simple for a small installation. Horizontal scaling adds another application instance behind a reverse proxy, but it requires shared session state, shared file storage, health checks, and often a separate database strategy.

The proposed architecture has a clear path for both. The gateway can distribute requests to multiple application VMs. The database can move to a managed PaaS service or a replicated cluster. File collaboration can move to an object store or SaaS provider. These changes reduce the single-host bottleneck, but they also add cost, identity integration, monitoring, and data-migration complexity.

- Stage 1: resize the existing VMs using measured utilization.
- Stage 2: separate application statelessness from database and file state.
- Stage 3: add a second application VM and health-checked routing.
- Stage 4: adopt managed PaaS database or SaaS collaboration when operational effort is the limiting factor.
- Stage 5: introduce multi-host virtualization and tested failover for critical services.

6.5 COST ANALYSIS

Cost is more than the purchase price of a server. It includes hardware, power, cooling, internet connectivity, software subscriptions, storage growth, backup media, maintenance time, training, and the cost of downtime. Virtualization reduces duplicated hardware for small workloads, but it does not make administration free.

The table below is a planning model, not a quotation. Replace the illustrative values with local prices before using it for a procurement decision. The model is useful because it exposes the trade-off between predictable subscriptions and self-managed infrastructure effort.

COST COMPONENT	TRADITIONAL SEPARATE SERVERS	VIRTUALIZED PRIVATE HOST	PUBLIC / MANAGED CLOUD
Hardware	High: several physical systems	Medium: one capable host + backup	Low upfront
Power and cooling	High	Lower through consolidation	Provider responsibility
Operations	Many OS instances	Fewer hosts, several guests	Subscription includes more platform work
Scaling	Procure new hardware	Resize or add host	Elastic but usage-priced
Failure domain	Several independent servers	One host can affect all VMs	Depends on tier and region
Best financial fit	Stable, dedicated loads	Small institution and lab	Variable demand / low operations team

6.6 ACCESSIBILITY AND USABILITY

Accessibility is part of availability. A platform that is reachable but difficult to use can still prevent students from completing academic work. The browser interface should use readable contrast, keyboard-friendly controls, clear status messages, meaningful link text, and responsive layouts. Documents should support accessible headings, alternative text for meaningful images, and predictable download behavior.

The infrastructure should also accommodate uneven connectivity. Lightweight pages, resumable uploads where possible, clear retry messages, and well-defined file-size limits reduce frustration. SaaS or CDN services may improve geographic reach, but they should not be introduced without checking data residency, identity, and cost implications.

AREA	DESIGN CHECK	EVIDENCE
Visual	Black text, readable size, sufficient contrast	Manual review / screenshot
Keyboard	Focus order and actions are usable	Keyboard walkthrough
Connectivity	Upload failure gives recovery guidance	Controlled retry test
Content	Headings, labels and file names are clear	Sample course page
Device	Works on desktop and mobile browser	Two viewport screenshots

7. SECURITY, COMPARISON AND REFLECTION

The proposed platform improves resource utilization and manageability by consolidating services, but consolidation increases the importance of the host and its backups. A balanced evaluation must therefore compare traditional and virtualized infrastructure across multiple dimensions rather than presenting virtualization as an unconditional improvement.

DIMENSION	TRADITIONAL PHYSICAL SETUP	PROPOSED VIRTUALIZED CLOUD SETUP	REFLECTION
Utilization	One service may leave hardware idle	Multiple services share host capacity	Better average utilization if monitored
Scalability	New hardware procurement	Resize, clone or migrate VMs	Faster but bounded by host
Accessibility	Often campus-bound	HTTPS, VPN and SaaS options	Broader access increases security duty
Cost	More hardware and power	Consolidated host plus operations	Savings depend on staffing and uptime
Security	Physical boundaries are clear	Logical isolation needs correct controls	Segmentation and patching are essential
Collaboration	Files and messages fragmented	Shared workspaces and versions	Better coordination with permissions

7.1 SECURITY OPERATIONS AND MONITORING

Security must continue after deployment. The administrator should maintain a patch calendar, review failed logins, check unusual file sharing, monitor disk growth, rotate credentials, and test restoration. Monitoring alerts should be actionable: a high CPU alert should name the VM and duration, while a backup alert should state whether the job failed or merely ran longer than expected.

Logs should be retained according to institutional policy and should avoid unnecessary personal data. Access to logs is itself controlled. A small environment can begin with systemd journal, firewall logs, application logs, and a metrics agent, then forward selected events to a central system as the institution grows.

- Daily: service health, backup status, disk usage, failed jobs.
- Weekly: pending updates, failed login review, restore sample, certificate expiry.
- Monthly: role and group review, capacity trend, incident drill, documentation update.
- After change: run smoke tests, record version, capture evidence, and update rollback notes.

7.2 LIMITATIONS AND CHALLENGES

The first limitation is the single-host demonstration. It proves virtualization and service separation, but it does not prove high availability. The second is the difference between an illustrative architecture and a production service: production requires capacity history, formal identity integration, support processes, service-level targets, and tested disaster recovery.

Other challenges include limited hardware, variable internet quality, staff training, licensing or SaaS subscription costs, privacy obligations, vendor lock-in, data migration, and user adoption. These challenges do not invalidate the design; they identify the decisions that must be made before a real migration.

CHALLENGE	EFFECT	MITIGATION
Limited host resources	Slow guests or swapping	Reduce scope; measure and resize
Single host failure	Many services unavailable	Second host; tested backups
Complex permissions	Accidental disclosure	RBAC templates and reviews
Provider lock-in	Harder migration	Export tests and open formats
Low user adoption	Workarounds and duplication	Training and simple workflows
Weak connectivity	Interrupted remote use	Caching, retry, local fallback

7.3 RISK REGISTER AND MITIGATION

RISK	LIKELIHOOD	IMPACT	RESPONSE / OWNER
Host hardware failure	Medium	High	Maintain separate backup and restore on standby host / Admin
Credential compromise	Medium	High	MFA, key-based SSH, revocation process / IAM owner
Full disk	Medium	High	Quotas, alerts, retention policy / Operations
Backup silently fails	Medium	High	Job monitoring and restore drill / Admin
Wrong file sharing scope	Medium	High	Private defaults and access review / Faculty
Service overload at deadline	Medium	Medium	Capacity test and temporary scale-up / Admin
Unpatched package	Medium	High	Patch window and vulnerability review / Admin
Documentation drift	High	Medium	Change checklist and repository review / Student team

7.4 ETHICAL, PRIVACY AND GOVERNANCE CONSIDERATIONS

Academic data includes identities, submissions, grades, communications, and sometimes sensitive personal information. The platform should collect only what it needs, restrict access by role, protect data in transit, define retention, and provide a process for correcting or removing records. A student's assignment should not become publicly accessible merely because a sharing link was convenient.

Governance also covers ownership and accountability. The institution should name a service owner, data owner, backup owner, and incident contact. Vendor terms and local regulations must be reviewed by the institution; this academic report does not certify compliance with any specific law or framework.

- Data minimization: store only required academic and operational information.
- Purpose limitation: do not reuse submissions or identity data without authorization.
- Transparency: explain who can view, edit, download, or restore a document.
- Accountability: keep audit records for privileged changes and access reviews.
- Responsible automation: validate permissions and backups before scaling the service.

7.5 OVERALL EVALUATION AGAINST RUBRIC

The report is structured to address the assignment's excellent-level indicators. It identifies requirements and maps them to all three service models; compares benefits, limitations, cost, scalability, accessibility, and security; defines VMs, resources, services and network connectivity; documents deployment and communication tests; explains secure remote access and roles; and reflects on traditional infrastructure, collaboration, challenges, and learning outcomes.

RUBRIC AREA	WHERE ADDRESSED	EVIDENCE TO ATTACH
CO1: service models	Chapters 1–2; Tables 1–3	Requirement map and comparison
CO2: architecture	Chapter 3; Figures 1 and 3	Architecture and network screenshots
CO2: VM deployment	Chapter 4; Table 4	Hypervisor and resource screenshots
CO2: communication	Chapter 6; Tests T02–T03	curl, nc, service status
CO3: cloud access	Chapter 5	HTTPS, SSH or VPN evidence
CO3: roles/security	Chapter 5; Figure 4–5	Permission tests and logs
Analysis/reflection	Chapters 6–7	Metrics, cost, risks, conclusion

8. EVIDENCE INDEX AND SUBMISSION CHECKLIST

The final submission should combine this report with authentic implementation evidence. Number each screenshot using the evidence ID, keep the image readable, and add one sentence explaining what it proves. If the institution requires a specific cover format, update the placeholders on page 1 without changing the technical sections.

DELIVERABLE FROM ASSIGNMENT	REPORT LOCATION	FINAL ATTACHMENT / ACTION
1. Architecture Design	Chapter 3; Figures 1–3	Keep diagrams and explain data paths
2. Virtualized Data Center Implementation & Results	Chapter 4 and 6	Attach actual VM and test evidence
3. Cloud Service Model Analysis	Chapter 2	Retain comparison and mapping tables
4. Screenshots / Evidence	Chapter 4.7 and this page	Insert E-01 to E-08 screenshots
5. Comparison and Analysis Report	Chapter 6 and 7	Check reflection and limitations
6. GitHub Upload	Page 48	Upload safe documentation and scripts

NOTE: *Before printing, replace all blank institutional fields, record actual test results, and remove any example domain names or placeholder values that are not used.*

8.1 GITHUB UPLOAD AND REPOSITORY GUIDE

GitHub provides a visible version history for the design, scripts, diagrams, and test documentation. Create a repository with a descriptive name such as cloud-virtualized-collaboration-platform. Do not upload passwords, API keys, private SSH keys, student records, database dumps, or unredacted screenshots. Use a .gitignore file and example configuration files with placeholder values.

The upload should make reproduction easy: a new reader should know the architecture, prerequisites, startup order, test commands, expected evidence, and limitations. Commit in meaningful stages such as docs, VM plan, service configuration, tests, and final evidence index.

PATH	CONTENT
README.md	Purpose, architecture, prerequisites, deployment order, tests
docs/report.docx	This report or a final exported copy
docs/diagrams/	Architecture and network diagrams
infra/vm-inventory.csv	Names, roles, resources; no secrets
scripts/	Installation and smoke-test scripts
evidence/README.md	Evidence index and screenshot naming rules
.gitignore	Secrets, dumps, temporary files, local VM disks

```
git init
git add README.md docs/ infra/ scripts/ .gitignore
git commit -m "Add cloud collaboration platform design"
git branch -M main
git remote add origin https://github.com/<account>/<repository>.git
git push -u origin main
```

NOTE: *Replace the remote URL with the student's own repository. Do not paste credentials into the repository or into this report.*

8.2 LEARNING OUTCOMES AND SDG MAPPING

CO1 is demonstrated through requirement analysis, service-model classification, and comparison of responsibility, cost, scalability, accessibility, and security. CO2 is demonstrated through the virtualized data-centre architecture, VM inventory, resource configuration, network topology, service communication, and deployment evidence. CO3 is demonstrated through secure cloud access, user roles, permissions, collaboration workflows, remote management, and operational controls.

The work also supports the mapped Sustainable Development Goals. SDG 4 is addressed by improving the digital infrastructure available for quality education. SDG 8 is supported by efficient resource utilization and digital work practices. SDG 9 is reflected in resilient infrastructure, innovation, virtualization, and a pathway from a small laboratory to a scalable service.

OUTCOME / SDG	DEMONSTRATED BY	REFLECTION
CO1	Chapters 1–2	Select service model according to responsibility and need
CO2	Chapters 3–4	Build and connect a multi-VM virtual data centre
CO3	Chapters 5–6	Secure access, collaborate, test and recover
SDG 4	Learning application and shared content	Access supports academic continuity
SDG 8	Consolidation and collaboration	Less duplication and more productive work
SDG 9	Virtualized infrastructure and scaling path	Innovation with maintainable foundations

CONCLUSION AND FUTURE ENHANCEMENTS

This report has designed a cloud-based virtualized collaboration platform for a small engineering college and connected the design to the assignment's three course outcomes. The solution consolidates application hosting, file sharing, database services, monitoring, and backup into role-specific VMs. Logical segmentation protects service and management traffic, while HTTPS, SSH controls, RBAC, patching, logs, and backups establish a defensible operational baseline.

The central finding is that cloud computing is a set of responsibility choices, not only a location for servers. IaaS is valuable for control and virtualization learning, PaaS reduces platform administration for applications and databases, and SaaS delivers mature collaboration features quickly. A mixed architecture is therefore more realistic than selecting one model for every requirement.

The next steps are to execute the deployment, capture real evidence, measure resource utilization under a representative workload, perform a restore drill, and refine the design using those results. Future enhancements include a second host, managed database, application replicas, stronger identity federation, object storage, centralized logging, automated configuration, and formal disaster-recovery testing.

- Immediate: complete the VM deployment and replace evidence placeholders with screenshots.
- Short term: automate installation and add health checks to the repository.
- Medium term: introduce second-host recovery and managed services where they reduce risk.
- Long term: integrate institutional identity, capacity forecasting, and formal governance.

REFERENCES

1. Mell, P., and Grance, T., "The NIST Definition of Cloud Computing," NIST Special Publication 800-145, National Institute of Standards and Technology, September 2011. Available at: https://nvlpubs.nist.gov/nistpubs/Legacy/SP/nistspecialpublication800-145.pdf
2. Jansen, W., and Grance, T., "Guidelines on Security and Privacy in Public Cloud Computing," NIST Special Publication 800-144, National Institute of Standards and Technology, December 2011. Available at: https://nvlpubs.nist.gov/nistpubs/Legacy/SP/nistspecialpublication800-144.pdf
3. Badger, M. L., Grance, T., Patt-Cornell, M., and Voas, J., "Cloud Computing Synopsis and Recommendations," NIST Special Publication 800-146, National Institute of Standards and Technology, May 2012. Available at: https://nvlpubs.nist.gov/nistpubs/Legacy/SP/nistspecialpublication800-146.pdf
4. International Organization for Standardization, "Information Technology – Cloud Computing – Vocabulary," ISO/IEC 17788:2014, International Organization for Standardization, 2014.
5. International Organization for Standardization, "Information Technology – Cloud Computing – Reference Architecture," ISO/IEC 17789:2014, International Organization for Standardization, 2014.
6. Oracle Corporation, "Oracle VM VirtualBox User Manual," Oracle VirtualBox Documentation. Available at: https://www.virtualbox.org/manual/
7. Ubuntu Documentation Team, "Ubuntu Server Documentation," Canonical Ltd. Available at: https://ubuntu.com/server/docs
8. PostgreSQL Global Development Group, "PostgreSQL Documentation," PostgreSQL Official Documentation. Available at: https://www.postgresql.org/docs/
9. Docker Inc., "Docker Documentation: Get Started with Docker," Docker Documentation. Available at: https://docs.docker.com/get-started/
10. Nextcloud GmbH, "Nextcloud Server Administration Manual," Nextcloud Documentation. Available at: https://docs.nextcloud.com/server/latest/admin_manual/
11. Open Web Application Security Project, "OWASP Application Security Verification Standard," OWASP Foundation. Available at: https://owasp.org/www-project-application-security-verification-standard/
12. Open Web Application Security Project, "OWASP Top 10: The Ten Most Critical Web Application Security Risks," OWASP Foundation. Available at: https://owasp.org/www-project-top-ten/
13. GitHub, "GitHub Documentation: Managing Repositories," GitHub Docs. Available at: https://docs.github.com/en/repositories

14. GitHub, “Ignoring Files: Creating a .gitignore File,” GitHub Docs. Available at: https://docs.github.com/en/get-started/getting-started-with-git/ignoring-files
15. Amazon Web Services, “AWS Well-Architected Framework,” Amazon Web Services Documentation. Available at: https://docs.aws.amazon.com/wellarchitected/latest/framework/welcome.html
16. Microsoft Azure, “Azure Architecture Center,” Microsoft Learn. Available at: https://learn.microsoft.com/en-us/azure/architecture/
17. Google Cloud, “Google Cloud Architecture Framework,” Google Cloud Documentation. Available at: https://cloud.google.com/architecture/framework
18. Kavis, M. J., *Architecting the Cloud: Design Decisions for Cloud Computing Service Models*, Wiley, 2014.
19. Erl, T., Puttini, R., and Mahmood, Z., *Cloud Computing: Concepts, Technology and Architecture*, Prentice Hall, 2013.
20. Buyya, R., Broberg, J., and Goscinski, A. M., *Cloud Computing: Principles and Paradigms*, Wiley, 2011.
21. Stallings, W., *Foundations of Modern Networking: SDN, NFV, QoE, IoT, and Cloud*, Addison-Wesley Professional, 2016.